

Table 2. Implicit Intermediate Representations and Implicit Decision Making

Classifications		Papers
Fundamental Methods	Sampling	[156]
	Graph Learning	[8, 42, 64, 68, 166, 194]
	Security	[85]
Logic and knowledge processing	Logical Reasoning	[3, 30, 49, 65, 75, 112, 116, 124, 140, 178, 186, 191, 202, 208, 209]
	Probabilistic Reasoning	[2, 9, 77–79, 130, 131, 133, 151, 170]
	Neural Automata	[182]
	Explainability	[63]
	Knowledge processing	[52, 114, 129]
Application	Question Answering	[24, 33, 123, 144, 207]
	Programming	[61, 62, 87]
	Education	[90, 91]
	Fault diagnosis	[154]
	Chemical	[158]
	Link prediction	[150]
	Emotion analysis	[23]
Mathematics/Symbolic Regression		[14, 21, 28, 31, 35, 76, 109, 110, 120, 136, 145]
Natural language processing		[48, 95, 104, 161]
Computer vision		[46, 50, 135]
Reinforcement learning		[47, 67, 81, 198]

Lemos et al. [114] proposed a neural symbolic model for relational reasoning and link prediction on knowledge graphs. The model takes a subset of the knowledge graph as input and then uses two learned embedding layers to map entity types and relationships into real-valued vector space to initialize the representation of entities and relationships so that entity types and relationships passing through the embedding layer are converted into real-valued vectors that capture the underlying semantic features of entity types and relationships. Then, the graph neural network is used to update the vector representation of entities and relationships iteratively, and through the message-passing mechanism of GNN(Graph Neural Network), each entity, which is represented by a node and relationship, which is represented by an edge in the graph exchange information and update their adjacency relationships. Representation captures complex dependencies between entities and relationships, and each entity and relationship has a unique identity in the symbolic logic graph. Finally, a multilayer perceptron is used to decode the vector representation of the target relationship and convert it into the predicted relationship type. This representation is a symbolic one that the neural network learns as a real-valued vector and converts to symbolic logic form to bridge the gap from data-driven feature extraction to symbolic logic reasoning. A core challenge faced by this research is the need for more consistency in the representation form. That is, while the neural network uses real-valued vectors to represent entities and relationships and captures a large amount of unstructured information in this form, the symbolic logic part relies on clearly defined symbols and rules to perform reasoning. This inconsistency necessitates the use of an intermediate representation to bridge the two. In this study, the author uses embedding vectors as an intermediate representation to bridge the deep learning feature expression and symbolic logic. However, these high-dimensional and dense vectors need to be more explicit and directly readable, which makes it impossible to observe and verify their correctness directly. At the same time, the decision-making logic of this method is also implicitly expressed through the weights and activation functions of the

neural network, so although the model can perform complex inference tasks, it is difficult to understand and examine the specific reasons why the model makes a particular prediction.

Ahmed et al. [2] proposed a method of assuming constraint decomposition to solve probabilistic reasoning problems conditioned on the characteristics of network learning. This paper implements the process through a semantic reinforcement approach, i.e., building logic circuits, calculating the probabilities of logic constraints, and using the probabilities output by neural networks to estimate the likelihood of different logic states. Semantic reinforcement methods assume that individual sub-constraints in a logical constraint are independent of each other, so the probability of entire constraints can be approximated by multiplying the probabilities of its sub-constraints. Therefore, basic logic operation units such as AND and OR gates are implemented by constructing logical constraint calculation graphs. This construction process follows structural rules such as decomposability and determination. Therefore, these logic circuits can effectively calculate each probability of satisfying logical constraints with the given input. At the same time, based on probabilistic decomposition, this method quantifies how strong the dependence between two sub-constraints is under the given neural network characteristics by calculating Conditional Mutual Information. It uses this to evaluate which sub-constraints are independent assumptions that may lead to inaccuracies in probability calculations. The accuracy of overall probability calculation can be further improved by gradually relaxing the independence assumption between these sub-constraints and merging them into more complex constraints. Throughout the process, features in the form of real-valued vectors are extracted from the input data through neural networks. However, they do not directly correspond to the discrete symbolic representation used by symbolic logic. Therefore, to bridge this gap, neural networks need to convert the extracted features into an intermediate representation by knowledge compilation technology, which can compile logical formulas into calculable circuit structures. However, it is implicit and cannot be directly observed and verified. Second, the overall decision logic is implicitly expressed through the weights and activation functions of the neural network and is not entirely transparent to external observers. Although computing conditional mutual information and performing semantic reinforcement provides a way to iteratively improve the decision logic to optimize the processing of logical constraints, the explainability of overall decision process still needs improvement.

Marconato et al. [129] proposed a neural symbolic continuous learning method and an optimization strategy, COOL(concept-level continual learning), to solve the catastrophic forgetting problem in a series of neural symbolic tasks. First, the method extracts features from visual data. It then uses a separate neural network to extract high-level concepts from these features, such as color and shape in the CLEVR(Compositional Language and Elementary Visual Reasoning) dataset or number recognition in the addition task in the MNIST(Modified National Institute of Standards and Technology database) dataset, which are then generated in final predictions or decisions based on the constraints of given prior knowledge. In this process, symbolic rules exist as explicit mathematical and logical expressions and serve as prior knowledge K to define the relationship between concepts and the mapping rules between concepts and predicted results. When faced with new tasks, this method adapts to the new concept representation by updating the neural network, modifying or extending the logic rules to reflect the new knowledge, and achieving continuous learning based on the new concept representation. After the neural network extracts from the visual input, these features exist as vectors, and this method uses another neural network as an intermediate representation to convert these vectors into visual concepts, as logical symbols cannot process them further. However, the intermediate representation is implicitly expressed through weights and activation functions; they cannot be directly observed or tested. So, even though this method's reasoning and prediction process is explicitly based on mathematical formulas or logical expressions, it still needs to be fully interpretable.

Furlong and Eliasmith [78] proposed a method for realizing probability calculation through VSA (vector symbolic architecture) and SSP(spatial semantic pointers) applied to cognitive models and neural networks. This method usually takes observation data or features as input data, such as images, sounds, or other types of sensor data. These observations are first converted into points as SSPs in a high-dimensional vector space by converting data into continuous space, where each vector represents an observation. Then, use the bundling, binding, unbundling, and other operations defined in VSA to perform abstract symbolic logic processing on the encoded data, such as calculating unified distribution, conditional distribution, entropy, and mutual information, etc., and executing calculations of conditional probabilities through unbundling operations, or estimating entropy or mutual information through a specific neural network structure, which means that probability calculations can be performed directly on the SSP representation through the above symbolic operations, based on this probability information, decision logic or prediction can be constructed, such as selecting actions with the highest expected reward, updating belief states based on current observations, or other forms of high-level cognitive tasks. This method can support constructing models that simulate human or animal cognitive processes such as decision-making, learning, and memory in handling uncertainty and probabilistic reasoning. It is worth mentioning that this method uses a neural network to map input information into high-dimensional vectors and uses logical symbolic methods to process these features directly. Therefore, although these two expressions are the same, the method can only be classified as low explainable, as the logical reasoning operations are in the implicit space.

In this category, the form of intermediate representation and the inference or prediction process partly depend on the neural network's output. Therefore, even though part of the process may be combined with an explicit logical symbolic process, it still belongs to low explainability as all efforts have not outstripped the black box effect of neural networks.

3.2 Partially Explicit Intermediate Representations and Partially Explicit Decision Making(prediction)

As Table 3 shows, this category involves 110 neuro-symbolic AI studies, and they have three common characteristics. First, they all use neural networks to extract features from data. However, the representations of these embeddings cannot be directly processed by symbolic logic, so they need an intermediate representation to fill the gap. Secondly, most intermediate representations are symbolic logic expressions, mathematical expressions, structured programs, logic circuits, probability distributions, virtual circuits, and virtual machine instructions. These representations are partially explicit and human-readable. Third, decision logic combines implicit representations from neural networks and explicit representations of symbolic logic, so it is partially explicit and readable.

Petersen et al. [143] proposed a DSR(Deep Symbolic Regression) method to recover mathematical expressions from data. This method first represents each expression as a node sequence through symbolic expression trees, including mathematical operators such as addition, subtraction, multiplication, and division and operands such as x and constants. A RNN(Recurrent Neural Network) is then used to predict the next operator or operand based on the existing sequence. In this method, RNN predicts new nodes based on existing expression trees, and logical symbols exist as expression trees of mathematical expressions. The tree structure represents the logical relationship between operators and operands, such as which two numbers are added together or which number is divided by another number. Each time the RNN generates a complete expression, the expression is used to calculate the degree of fit on a specific data set, such as by calculating the difference between the expression and the actual data, and the fitting is feedback to the RNN to guide the adjustment of the subsequent expression generation process. In DSR, the weights, activation functions, and currently processed sequences inside the RNN act as an implicit intermediate representation, although this representation is not directly oriented to the end user, nor is it in any explicit or structured way. The symbolic form exists, but it connects the input sequence and the output prediction and is finally output explicitly in the form of a specific symbol in a